NLP Task-1
Financial Sentiment Analysis

A comprehensive study of contextual embeddings and classification pipelines using DistilBERT on the FinancialPhraseBank dataset

---

Author: Shaurya Handu

Dataset	FinancialPhraseBank-v1.0 (Sentences_50Agree)
Model	`distilbert-base-uncased`
Framework	PyTorch + HuggingFace Transformers
Task	Three-class Sentiment Classification

March 29, 2026

Contents

1 Introduction **2**
1.1 Objectives . . . 2

2 Dataset: FinancialPhraseBank-v1.0 **2**
2.1 Overview . . . 2
2.2 Format and Loading . . . 2
2.3 Class Distribution . . . 3
2.4 Train/Test Split . . . 3

3 Project Structure **3**

4 Contextual Encoder (`encoder.py`) **3**
4.1 Model: DistilBERT . . . 4
4.1.1 Why Contextual Embeddings? . . . 4
4.2 Pooling Strategies . . . 4
4.2.1 CLS Pooling . . . 4
4.2.2 Mean Pooling . . . 4
4.3 Disambiguation Probe . . . 4

5 Sentiment Pipeline (`sentiment_pipeline.py`) **5**
5.1 Overview . . . 5
5.2 MLP Classifier Architecture . . . 5
5.3 Setup 1: Frozen CLS + Logistic Regression . . . 5
5.4 Setup 2: Frozen Mean Pooling + Logistic Regression . . . 5
5.5 Setup 3: Frozen Mean Pooling + MLP . . . 6
5.6 Setup 4: End-to-End Fine-Tuning . . . 7

6 Baselines **8**
6.1 VADER Baseline . . . 8
6.2 CBOW Reference . . . 8

7 Results Summary and Analysis **8**
7.1 Overall Comparison . . . 8
7.2 Key Observations . . . 9
7.3 Confusion Matrix Analysis . . . 9

8 Technical Implementation Details **9**
8.1 Tokenisation . . . 9
8.2 Hardware and Inference . . . 9
8.3 Dependencies . . . 10
8.4 Evaluation Metric . . . 10

9 Conclusion **10**
9.1 Future Work . . . 11

1 Introduction

Financial sentiment analysis is the task of automatically identifying the emotional tone or opinion expressed in financial text – determining whether a piece of news, earnings statement, or analyst commentary conveys a *positive*, *neutral*, or *negative* sentiment toward a financial entity.

This report documents the full NLP Task-1 pipeline as implemented in a Kaggle Jupyter notebook. The project explores four distinct modelling setups of increasing sophistication, ranging from simple frozen-encoder feature extraction with logistic regression to full end-to-end fine-tuning of a transformer model. All setups are evaluated against VADER and CBOW baselines on the FinancialPhraseBank dataset.

1.1 Objectives

- Extract contextual embeddings from financial text using DistilBERT.
- Compare CLS-token pooling versus mean-pooling strategies.
- Evaluate frozen-encoder setups against an end-to-end fine-tuned model.
- Benchmark all setups against rule-based (VADER) and shallow (CBOW) baselines.
- Quantify contextual disambiguation capability via a "bank" probe.

2 Dataset: FinancialPhraseBank-v1.0

2.1 Overview

The FinancialPhraseBank dataset [1] contains sentences manually annotated with sentiment labels by financial domain experts. The variant used here is `Sentences_50Agree.txt`, which includes only sentences where at least 50% of annotators agreed on the label.

2.2 Format and Loading

Each line in the raw file follows the pattern:

`"<sentence text>.@<label>"`

where `label` $\in$ {`negative`, `neutral`, `positive`}. The loader uses a regular expression to parse each line:

```
match = re.search(r'\.@(negative|neutral|positive)\s*$', line)
if match:
    sentiment = match.group(1)
    sentence  = line[:match.start()].strip()
    label     = label_map[sentiment]   # {negative:0, neutral:1,
   positive:2}
```

Listing 1: Dataset loading logic from `evaluate_sentiment.py`

Class	Label ID	Count	Proportion
Negative	0	587	12.4%
Neutral	1	2816	59.5%
Positive	2	1327	28.1%
Total	–	**4730**	100%

Table 1: FinancialPhraseBank class distribution (Sentences_50Agree)

2.3 Class Distribution

After loading, the dataset contains **4,730 sentences** with the following label distribution:

The dataset is **highly imbalanced**, with neutral sentences making up nearly 60% of samples. This motivates the use of **macro-averaged F1** as the primary evaluation metric, which weights each class equally regardless of support.

2.4 Train/Test Split

The data is split into an 80/20 train/test partition using stratified sampling to preserve class proportions:

$$|\text{Train}| = 3784 \quad (80\%)$$
$$|\text{Test}| = 946 \quad (20\%)$$

A fixed random seed of 42 ensures reproducibility.

3 Project Structure

The project is organized into three Python modules housed in the `financial-sentiment-analysis/` directory:

```
financial-sentiment-analysis/
    encoder.py                  # FinancialEncoder: embedding extraction
    sentiment_pipeline.py       # SentimentPipeline: Setups 1-4
    evaluate_sentiment.py       # CLI entry point and evaluation harness
    data/
    models/
    results/
```

- **encoder.py** (4,366 bytes): encapsulates DistilBERT loading, CLS/mean pooling, and a disambiguation probe.
- **sentiment_pipeline.py** (13,033 bytes): implements all four experimental setups plus the MLP classifier.
- **evaluate_sentiment.py** (6,694 bytes): CLI driver that loads data, invokes the pipeline, computes VADER baseline, and prints a results summary.

4 Contextual Encoder (`encoder.py`)

4.1 Model: DistilBERT

The backbone model is `distilbert-base-uncased`, a distilled version of BERT that retains approximately 97% of BERT's language understanding while being 40% smaller and 60% faster. It produces contextual token embeddings of dimension $d = 768$.

4.1.1 Why Contextual Embeddings?

Unlike static word vectors (Word2Vec, GloVe, CBOW), contextual embeddings assign different vector representations to the same word depending on its surrounding context. This is essential for financial text, where words like *bank*, *interest*, or *market* carry fundamentally different meanings in different contexts.

4.2 Pooling Strategies

Given a sequence of n tokens, the transformer produces a hidden state matrix $\mathbf{H} \in \mathbb{R}^{n \times d}$. To obtain a single sentence-level vector, two pooling strategies are implemented:

4.2.1 CLS Pooling

The `[CLS]` token is a special token prepended to every input. Its final hidden state is designed (after pre-training) to capture aggregate sequence-level information:

$$\mathbf{e}_{\text{CLS}} = \mathbf{H}_{[0,:]} \in \mathbb{R}^d \tag{1}$$

4.2.2 Mean Pooling

Mean pooling averages the hidden states of *all* non-padding tokens, weighted by the attention mask $\mathbf{m} \in \{0, 1\}^n$:

$$\mathbf{e}_{\text{mean}} = \frac{\sum_{i=1}^{n} m_i \cdot \mathbf{H}_{[i,:]}}{\sum_{i=1}^{n} m_i} \tag{2}$$

Mean pooling tends to produce more uniformly distributed representations because it incorporates information from every token, not just the CLS position.

4.3 Disambiguation Probe

The `FinancialEncoder` includes a built-in diagnostic called the **disambiguation probe**. It encodes the following two sentences:

1. "The bank raised interest rates." *(financial context)*
2. "She sat by the river bank." *(geographic context)*

The cosine similarity between the two embeddings is computed:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} \tag{3}$$

A **lower cosine similarity** indicates that the model assigns meaningfully different representations to the word *bank* in its two contexts – evidence of true contextual understanding rather than static lookup.

5 Sentiment Pipeline (`sentiment_pipeline.py`)

5.1 Overview

The `SentimentPipeline` class wraps the DistilBERT encoder and exposes four experimental setups. All setups share the same base model (`distilbert-base-uncased`) and the same 80/20 stratified split.

5.2 MLP Classifier Architecture

Setups 3 uses a purpose-built two-layer MLP:

$$\text{MLP}(\mathbf{x}) = W_2 \cdot \text{Dropout}(\text{ReLU}(W_1\mathbf{x} + b_1)) + b_2 \tag{4}$$

where $W_1 \in \mathbb{R}^{128\times256}$ and $W_2 \in \mathbb{R}^{3\times128}$. The architecture is:

$$\textbf{256} \rightarrow \textbf{ReLU} \rightarrow \textbf{Dropout(0.2)} \rightarrow \textbf{128} \rightarrow \textbf{3 (softmax)}$$

5.3 Setup 1: Frozen CLS + Logistic Regression

Description: The DistilBERT encoder is *frozen* (weights not updated). CLS embeddings ($d = 768$) are extracted for all training and test sentences. A multi-class Logistic Regression classifier (`max_iter=1000`, L2 regularisation) is trained on the resulting feature matrix.

Pipeline:

$$\text{Text} \xrightarrow{\text{DistilBERT (frozen)}} \mathbf{e}_{\text{CLS}} \in \mathbb{R}^{768} \xrightarrow{\text{Logistic Regression}} \hat{y} \in \{0, 1, 2\}$$

Strengths: Fast to train; no risk of catastrophic forgetting; interpretable linear boundary.
Weaknesses: Embeddings not optimised for the downstream task; CLS pooling may discard useful token-level information.

Result: Macro F1 = **0.7435**

Class	Precision	Recall	F1	Support
Negative	0.72	0.69	0.71	117
Neutral	0.82	0.86	0.84	563
Positive	0.71	0.65	0.68	266
Macro avg	0.75	0.74	0.74	946
Weighted avg	0.78	0.78	0.78	946

Table 2: Setup 1 classification report

5.4 Setup 2: Frozen Mean Pooling + Logistic Regression

Description: Identical to Setup 1 except that mean-pooled embeddings replace CLS embeddings. All other hyperparameters are unchanged.

Pipeline:

$$\text{Text} \xrightarrow{\text{DistilBERT (frozen)}} \mathbf{e}_{\text{mean}} \in \mathbb{R}^{768} \xrightarrow{\text{Logistic Regression}} \hat{y}$$

Key difference from Setup 1: Mean pooling incorporates every token's representation, potentially capturing richer semantic content than the single CLS vector.

Result: Macro F1 = **0.7583** (+0.0148 over Setup 1)

Class	Precision	Recall	F1	Support
Negative	0.74	0.68	0.71	117
Neutral	0.84	0.88	0.86	563
Positive	0.73	0.68	0.71	266
Macro avg	0.77	0.75	0.76	946
Weighted avg	0.80	0.80	0.80	946

Table 3: Setup 2 classification report

5.5 Setup 3: Frozen Mean Pooling + MLP

Description: Mean-pooled embeddings ($d = 768$) are first reduced to 256 dimensions via PCA, then passed to the two-layer MLP classifier. The MLP is trained for 10 epochs with Adam optimiser (lr=0.001) and cross-entropy loss; the DistilBERT encoder remains frozen.

Pipeline:

$$\text{Text} \xrightarrow{\text{DistilBERT (frozen)}} \mathbf{e}^{768}_{\text{mean}} \xrightarrow{\text{PCA}} \mathbf{z}^{256} \xrightarrow{\text{MLP}} \hat{y}$$

PCA rationale: Reducing from 768 to 256 dimensions removes noise, reduces overfitting risk, and makes the MLP's input manageable. PCA is fit on the training set and applied to both train and test.

Training curve:

Epoch	Cross-Entropy Loss
1	0.8166
2	0.5256
3	0.4269
5	0.3509
8	0.2763
10	0.2351

Table 4: Setup 3 MLP training loss progression

Result: Macro F1 = **0.7655** (+0.0220 over Setup 1)

Class	Precision	Recall	F1	Support
Negative	0.74	0.72	0.73	117
Neutral	0.84	0.88	0.86	563
Positive	0.74	0.67	0.71	266
Macro avg	0.77	0.76	0.77	946
Weighted avg	0.80	0.80	0.80	946

Table 5: Setup 3 classification report

5.6 Setup 4: End-to-End Fine-Tuning

Description: Both the DistilBERT encoder *and* a linear classification head are trained jointly from scratch on the financial sentiment task. This allows the transformer's weights to adapt to the specific vocabulary and style of financial language.

Architecture:

$$\text{Text} \xrightarrow{\text{DistilBERT (trainable)}} \mathbf{e}^{768}_{\text{mean}} \xrightarrow{W \in \mathbb{R}^{3\times 768}} \text{logits} \xrightarrow{\text{softmax}} \hat{y}$$

Hyperparameters:

- Optimiser: AdamW
- Learning rate: 2×10^{-5} (with linear warmup scheduler)
- Batch size: 16
- Epochs: 5
- Max sequence length: 512 tokens

The learning rate scheduler uses linear warmup to zero warm-up steps and decays to zero over the total training steps, which stabilises early training and prevents large gradient updates from corrupting the pre-trained weights.

Training curve:

Epoch	Cross-Entropy Loss
1	0.5489
2	0.2549
3	0.1386
4	0.0750
5	0.0419

Table 6: Setup 4 fine-tuning loss progression

The loss drops sharply from epoch 1 to 2 and converges near zero by epoch 5, demonstrating effective adaptation to the financial domain.

Result: Macro F1 = **0.8529** (the best setup overall)

Class	Precision	Recall	F1	Support
Negative	0.81	0.91	0.85	117
Neutral	0.91	0.87	0.89	563
Positive	0.81	0.82	0.81	266
Macro avg	0.84	0.87	0.85	946
Weighted avg	0.87	0.86	0.86	946

Table 7: Setup 4 classification report

6 Baselines

6.1 VADER Baseline

VADER (Valence Aware Dictionary and sEntiment Reasoner) is a lexicon- and rule-based sentiment tool designed for social media text. For each sentence, the compound score $c \in [-1, 1]$ is computed and thresholded:

$$\hat{y} = \begin{cases} 0 & \text{if } c < -0.1 \quad \text{(negative)} \\ 2 & \text{if } c > +0.1 \quad \text{(positive)} \\ 1 & \text{otherwise} \quad \text{(neutral)} \end{cases} \tag{5}$$

VADER Macro F1 = 0.4796

VADER performs poorly because it was not designed for financial text. Financial language is often understated and domain-specific (e.g., "the company restructured its debt" is negative, but VADER sees neutral words).

6.2 CBOW Reference

A Continuous Bag-of-Words (CBOW) + Logistic Regression reference score of **0.68–0.74** is reported. CBOW produces static, context-insensitive word embeddings averaged into sentence vectors, making it a natural comparison point for contextual embeddings.

7 Results Summary and Analysis

7.1 Overall Comparison

Setup	Description	Macro F1	Δ vs CBOW
VADER Baseline	Rule-based (no training)	0.4796	−0.20 to −0.26
CBOW Reference	Static embeddings + LR	0.68–0.74	–
Setup 1	Frozen CLS + Logistic Regression	0.7435	+0.00 to +0.06
Setup 2	Frozen Mean Pool + Logistic Regression	0.7583	+0.02 to +0.08
Setup 3	Frozen Mean Pool + MLP	0.7655	+0.03 to +0.09
Setup 4	**End-to-End Fine-Tuning**	**0.8529**	+0.11 to +0.17

Table 8: Summary of all setups and baselines

7.2 Key Observations

1. Fine-tuning far outperforms frozen approaches. Setup 4 achieves a macro F1 of 0.8529, a **+9.74 percentage point** improvement over the best frozen setup (Setup 3). Allowing the transformer to update its attention patterns and feed-forward weights specifically for financial language is the single largest source of performance gain.

2. Mean pooling beats CLS pooling. Setup 2 (mean pooling) achieves F1 = 0.7583 vs. Setup 1's F1 = 0.7435. By averaging over all token representations, mean pooling captures distributional semantics more holistically than the single CLS vector.

3. MLP provides marginal gain over logistic regression on frozen features. Setup 3 (+0.0072 vs. Setup 2) shows that a non-linear classifier can exploit subtle structure in the embedding space, but the gain is small when the encoder itself is not adapted to the task.

4. Contextual embeddings dramatically outperform VADER. The +27 percentage point gap between any DistilBERT-based setup and VADER confirms that rule-based systems are insufficient for financial sentiment, where tone is highly domain-specific and context-dependent.

5. Minority class (negative) improves most with fine-tuning. Negative-class recall jumps from 0.69 (Setup 1) to 0.91 (Setup 4), a 22-point improvement. Fine-tuning allows the model to learn domain-specific negative markers that are invisible to frozen embeddings.

7.3 Confusion Matrix Analysis

In Setup 4, the dominant error mode is **positive→neutral** misclassification (42 samples), followed by **neutral→positive** (51 samples). The boundary between these two classes is genuinely ambiguous in financial language, where mild optimism and factual reporting are stylistically similar.

8 Technical Implementation Details

8.1 Tokenisation

All text is tokenised using the HuggingFace `AutoTokenizer` with:

- Padding to the longest sequence in each batch
- Truncation at 512 tokens (DistilBERT's maximum context window)
- WordPiece subword tokenisation

8.2 Hardware and Inference

The code automatically detects and uses CUDA if available:

```
device = torch.device("cuda" if torch.cuda.is_available() else "
    cpu")
```

```
model.to(device)
```

Listing 2: Device detection

In the Kaggle environment, this typically maps to a Tesla P100 GPU, reducing embedding extraction time significantly compared to CPU inference.

8.3 Dependencies

Library	Purpose
`transformers >= 4.x`	DistilBERT tokeniser and model
`torch`	Tensor operations, autograd, training loop
`scikit-learn`	Logistic Regression, PCA, metrics
`nltk`	VADER sentiment analyser
`numpy`	Numerical array operations
`pandas`	Results tabulation

Table 9: Project dependencies

8.4 Evaluation Metric

All setups are evaluated using **macro-averaged F1-score**:

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^{C} \frac{2 \cdot P_c \cdot R_c}{P_c + R_c} \tag{6}$$

where $C = 3$ classes. Macro averaging is appropriate here because the dataset is imbalanced and all classes (including the minority negative class) are equally important in a real financial application.

9 Conclusion

This report presented a systematic study of four progressively sophisticated approaches to financial sentiment analysis on the FinancialPhraseBank dataset.

The central finding is that **end-to-end fine-tuning of DistilBERT** (Setup 4, F1 = 0.8529) substantially outperforms all frozen-encoder configurations and is the clear recommended approach for production deployment. The performance gap stems from the model's ability to adapt its internal representations to the specific linguistic patterns of financial reporting.

Among frozen setups, **mean pooling consistently outperforms CLS pooling**, confirming that sentence-level representations benefit from aggregating information across all token positions. Adding a non-linear MLP classifier (Setup 3) yields modest further gains over logistic regression.

The contextual nature of DistilBERT embeddings, illustrated by the disambiguation probe, provides a principled explanation for why these approaches substantially outperform both VADER (rule-based, F1 = 0.48) and CBOW (static embeddings, F1 = 0.68–0.74).

9.1 Future Work

- Fine-tuning a **domain-adapted** model such as FinBERT, which was pre-trained on financial corpora, would likely improve upon Setup 4.
- Applying **class-weighted loss** or oversampling techniques could further improve negative-class recall.
- Extending evaluation to the **100% agreement** variant of FinancialPhraseBank would provide insight into performance on unambiguous examples.
- Experimenting with **longer training** (10–15 epochs) and learning rate scheduling for Setup 4 may push performance higher.

References

[1] Malo, P., Sinha, A., Korhonen, P., Wallenius, J., & Takala, P. (2014). *Good debt or bad debt: Detecting semantic orientations in economic texts.* Journal of the Association for Information Science and Technology, 65(4), 782–796.

[2] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.* arXiv preprint arXiv:1910.01108.

[3] Hutto, C., & Gilbert, E. (2014). *VADER: A parsimonious rule-based model for sentiment analysis of social media text.* Proceedings of the International AAAI Conference on Web and Social Media, 8(1).

[4] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of deep bidirectional transformers for language understanding.* arXiv preprint arXiv:1810.04805.